

Project: Serverless ML API

07-projects/serverless-ml-api

Overview

A serverless ML API exposes model functionality through a managed API layer and event driven or on demand compute instead of continuously running servers. The design trades infrastructure simplicity against packaging and latency constraints.

It is a good pattern for intermittent usage, control plane operations around models, or lightweight inference that does not justify an always on fleet.

Core Concepts

- The typical stack is API Gateway plus Lambda, sometimes combined with S3, DynamoDB, Step Functions, or an external managed inference endpoint for heavier model execution.
- The key question is where the model runs. Very small models may execute inside Lambda, while larger models may require the API layer to dispatch to SageMaker, ECS, or an asynchronous backend.
- Security, request validation, quotas, and idempotency all sit close to the API boundary because public serverless endpoints are easy to invoke at scale.

How It Works

- A client sends a request to the API, which authenticates and validates the payload before invoking the function or orchestration path.
- The backend performs preprocessing, optional feature or state lookup, model invocation, and response shaping, then returns a result or a job reference.
- CloudWatch metrics and logs capture request count, latency, errors, and downstream dependency behavior so the system can be tuned over time.

AI and ML Relevance

- This project is useful for understanding which ML capabilities fit naturally into serverless boundaries and which do not.
- It also demonstrates how to expose prediction features with low operational overhead while preserving good access control and cost visibility.

Operational Notes

- Cold starts, package size, and downstream timeout chains are the main operational risks. Heavy frameworks or large model files can make the design fail its latency objective.
- Guard against abuse with throttling and auth because a public inference API can turn into an expensive denial of wallet event.
- Prefer asynchronous patterns when inference time or payload size approaches the practical limits of synchronous serverless execution.

What To Remember

- Serverless APIs are excellent when traffic is bursty and the unit of work is modest.
- Do not force a large real time model entirely into Lambda just because serverless looks simple on a diagram.
- The right design often mixes serverless control flow with a separate managed inference runtime.